Kotlin

Classes, objects and interfaces

Object-Oriented Programming · 2026/2027

Code, worked examples and exercises in the Kotlin Playground

Fernando Lucatelli Nunes

The route through the lesson

The main sequence covers six topics. The appendix develops the syntax used in the projects.

Preparation: read a program, distinguish types, declare functions and follow execution.

Classes: construct objects, store properties, define methods and control changes.

Data classes and enums: represent values and choose between a fixed set of alternatives.

Object and companion object: use a shared object and organise the construction of instances.

Interfaces: define common operations and run different implementations. A modular calculator brings the concepts together at the end.

One example, one file, one execution

Each code example identifies a program by its .kt file. That file contains all the declarations and a single main function.

When a program spans several slides, the title or explanation identifies the part shown. To run the program, copy the complete file.

The expected output appears beside the example and in an .out file. Compare both the content and the order of the lines.

Exercises have starter files in exercicios/ and solutions in resolucoes/. The solution appears on a later slide.

Kotlin Playground

Open https://play.kotlinlang.org and select Kotlin/JVM. Use a current stable version.

Delete the initial example and paste the complete contents of one .kt file from the package.

Press Run. Read the output and compare it with the expected output. Change one value and run the program again.

The examples in this lesson use data written into the program. You can try them without installing tools or entering input through the console.

TODO("message") compiles, but stops execution when called. In the exercises, replace it with the required implementation.

Reading Kotlin for the first time

CODE programas/P04_PrimeiroPrograma.kt

```kotlin
fun main() {
    // This line is a comment.
    val nome: String = "Ana"
    println(nome)
    println("Library")
}
```

OUTPUT

```
Ana
Library
```

fun declares a function. main is the entry point: execution begins in its body.

The empty parentheses indicate that this main takes no parameters. The braces delimit its body.

val declares nome. The colon introduces the type String. = associates the variable with the text in quotation marks.

println(...) calls a function with one argument and ends the output line. Instructions execute from top to bottom.

// begins a comment that continues to the end of the line. Indentation makes the code easier to read. Instructions do not need a final semicolon.

Variables and types

CODE programas/P05_Variaveis.kt

```
01 fun main() {
02     val limite: Int = 3
03     var quantidade: Int = 1
04     quantidade = quantidade + 1
05     val permitido: Boolean = quantidade <= limite
06     val grande: Long = 12
07     println(quantidade)
08     println(permitido)
09     println(grande)
10 }
```

OUTPUT

```
2
true
12
```

val prevents reassignment of a variable after initialisation. var allows a new assignment.

Int stores 32-bit integers. Long stores 64-bit integers. The L suffix explicitly marks a Long literal.

Boolean has the values true and false. <= compares the numbers and produces a Boolean.

In the assignment quantidade = quantidade + 1, the right-hand side is evaluated first, using the old value.

The type can be inferred: val limite = 3 also declares an Int. "3" would be text, with type String.

Expressions and integer arithmetic

CODE programas/P06_Aritmetica.kt

```kotlin
fun main() {
    val total = 95
    val horas = total / 60
    val minutos = total % 60
    println(horas)
    println(minutos)
    println(horas * 60 + minutos)
}
```

OUTPUT

```
1
35
95
```

/ between integers calculates the integer quotient. 95 / 60 is 1. The result is still an integer.

% calculates the remainder. 95 % 60 is 35 because 95 = 1 × 60 + 35.

* multiplies, + adds and - subtracts. Multiplication and division take precedence over addition and subtraction.

Parentheses group an expression: (2 + 3) * 4 is 20. The calculated value can be stored or passed to a function.

In Kotlin, -1 % 5 is -1. We will use explicit normalisation to obtain non-negative modular representatives.

Conditions and comparisons

CODE programas/P07_Condicoes.kt

```
01 fun main() {
02     val hora = 23
03     val valida = hora >= 0 && hora <= 23
04     if (valida) {
05         println("Time accepted")
06     } else {
07         println("Time rejected")
08     }
09     println(hora == 23)
10     println(hora != 23)
11 }
```

OUTPUT

```
Time accepted
true
false
```

if tests the condition in parentheses. When it is true, the first block executes. When it is false, the else block executes.

&& requires both conditions to be true. || requires at least one to be true. ! negates a Boolean.

&& and || evaluate the right-hand side only when it is needed to determine the result.

== tests equality and != tests inequality. = initialises or assigns a value. <, >, <= and >= compare order.

hora in 0..23 expresses the same inclusive range test. .. includes both endpoints, and in tests membership of the range.

Functions, parameters and results

CODE programas/P08_Funcoes.kt

```
01 fun totalMinutos(h: Int, m: Int): Int {
02     return h * 60 + m
03 }
04
05 fun main() {
06     val total = totalMinutos(1, 35)
07     println(total)
08 }
```

OUTPUT

```
95
```

In the declaration, h and m are parameters of type Int, separated by a comma. : Int after) gives the result type.

In the call totalMinutos(1, 35), the arguments 1 and 35 occupy the positions of h and m.

return passes the calculated value back to the caller and ends this execution of the function.

The call calculates 1 * 60 + 35, returns 95 and only then initialises total. The following line prints 95.

Declaring a function describes its work. Its body executes when the function is called. Functions can be declared directly in a file.

Expression bodies and Unit

CODE programas/P09_ExpressaoUnit.kt

```
01 fun totalMinutos(h: Int, m: Int): Int = h * 60 + m
02
03 fun mostrar(total: Int): Unit {
04     println(total)
05 }
06
07 fun main() {
08     mostrar(totalMinutos(1, 35))
09 }
```

OUTPUT

```
95
```

= after the signature introduces an expression body. That expression supplies the result directly, without return.

mostrar takes an Int and displays it. Unit indicates that the function does not provide a useful result to the caller.

For a function whose body is enclosed in braces, : Unit can be omitted. Its body still executes normally.

In the last line, the inner call finishes first. mostrar then receives 95 and prints it.

Calculating a result allows it to be reused in another calculation. println displays the value to the user.

Text, interpolation and the dot

CODE programas/P10_Textos.kt

```
fun main() {
    val nome = " Ana "
    val limpo = nome.trim()
    val minutos = 35
    println("Name: $limpo")
    println("Letters: ${limpo.length}")
    println("Total: ${60 + minutos}")
}
```

OUTPUT

```
Name: Ana
Letters: 3
Total: 95
```

A String is text enclosed in quotation marks. $limpo inserts the value of the variable into the message.

${...} inserts the result of an expression, such as limpo.length or 60 + minutos.

The dot accesses properties or calls functions on the value to its left. nome.trim() returns text without leading or trailing whitespace.

length is a property accessed without parentheses. Its value is the length of the text.

trim() returns another String. The variable nome still refers to the original text. We will now define our own types.

A class describes the objects we can create

CODE programas/C01_PrimeiroLivro.kt

```
01 class Livro(val id: String, val titulo: String)
02
03 fun main() {
04     val livro = Livro("L1", "Mensagem")
05     println(livro.id)
06     println(livro.titulo)
07 }
```

OUTPUT

```
L1
Mensagem
```

A class defines a type. Livro describes the data stored in each object of that type.

The parentheses after Livro declare the primary constructor. id and titulo receive arguments when we construct the object.

val makes these parameters read-only properties. Each object stores its own values.

Livro("L1", "Mensagem") creates an instance. livro holds a reference to that instance. Kotlin does not need the word new.

The dot selects a property: livro.id. A class without a body can end without braces.

Two copies have their own data

CODE programas/C02_DoisLivros.kt

```kotlin
class Livro(val id: String, val titulo: String)

fun main() {
    val primeiro = Livro("L1", "Mensagem")
    val segundo = Livro("L2", "Mensagem")
    println("${primeiro.id}: ${primeiro.titulo}")
    println("${segundo.id}: ${segundo.titulo}")
}
```

OUTPUT

```
L1: Mensagem
L2: Mensagem
```

Each constructor call creates an object. We use the same class to represent two physical copies of the book.

primeiro and segundo refer to different objects. Their titles can match while their identifiers differ.

${primeiro.id} evaluates the property access and inserts the result into the text. The braces are part of the interpolation.

The copy identifier is part of our model. The class alone does not yet prevent duplicate identifiers.

Constructor parameters and properties

CODE programas/C03_ParametroPropriedade.kt

```
01 class Rotulo(texto: String) {
02     val textoGuardado: String = texto
03 }
04
05 fun main() {
06     val rotulo = Rotulo("Library")
07     println(rotulo.textoGuardado)
08 }
```

OUTPUT

```
Library
```

In the constructor, texto has neither val nor var. It is a parameter used during initialisation.

The declaration val textoGuardado creates a property and stores the value received through texto.

Outside the class, rotulo.textoGuardado exists. rotulo.texto does not, because texto was not declared as a property.

Writing class Rotulo(val texto: String) would be another option: it would create the property texto directly.

A method uses the data of its object

CODE programas/C04_MetodoDescricao.kt

```
01 class Livro(val id: String, val titulo: String) {
02     fun descricao(): String {
03         return "$id: $titulo"
04     }
05 }
06
07 fun main() {
08     val livro = Livro("L1", "Mensagem")
09     val texto = livro.descricao()
10     println(texto)
11 }
```

OUTPUT

```
L1: Mensagem
```

A function declared inside a class is a method. descricao belongs to objects of type Livro.

Empty parentheses indicate that there are no parameters. : String declares the result type. return supplies that result.

Inside the method, id and titulo are the properties of the object on which the call was made.

livro.descricao() executes the method. Storing the result in texto does not display it on the screen: println performs that step.

this identifies the object receiving the call

CODE programas/C05_This.kt

```
01 class Contador(var valor: Int) {
02     fun definir(valor: Int) {
03         this.valor = valor
04     }
05 }
06
07 fun main() {
08     val contador = Contador(2)
09     contador.definir(7)
10     println(contador.valor)
11 }
```

OUTPUT

```
7
```

The constructor declares a var property named valor. var allows another integer to be assigned to it after construction.

The method also takes a parameter named valor. Inside the method, the unqualified name refers to that parameter.

this.valor selects the property of the current object. The assignment stores the received argument in that property.

contador.definir(7) changes the state of that counter. The method declares no useful result, so its return type is Unit.

Named arguments and default values

CODE programas/C06_Argumentos.kt

```kotlin
class Leitor(
    val nome: String,
    val ativo: Boolean = true
)

fun main() {
    val ana = Leitor("Ana")
    val rui = Leitor(ativo = false, nome = "Rui")
    println("${ana.nome}: ${ana.ativo}")
    println("${rui.nome}: ${rui.ativo}")
}
```

OUTPUT

```
Ana: true
Rui: false
```

In the constructor, = true defines the value used when the call omits the ativo argument.

Leitor("Ana") supplies nome and uses true for ativo. No other object affects that choice.

In the second call, ativo = false and nome = "Rui" are named arguments. Each name identifies the corresponding parameter.

Names allow arguments to be written in a different order. A default value is specified in the declaration, not in the call.

init checks data during construction

CODE programas/C07_InitRequire.kt

```
01 class Hora(val horas: Int, val minutos: Int) {
02     init {
03         require(horas >= 0 && horas <= 23) {
04             "The hour must be between 0 and 23."
05         }
06     }
07 }
08
09 fun main() {
10     val partida = Hora(23, 40)
11     println(partida.horas)
12 }
```

OUTPUT

```
23
```

init delimits a block that executes during construction. Here it checks horas. On the next slide, we will also validate minutos.

>= and <= compare integers. && requires both conditions to be true.

require takes a Boolean condition. When it is false, it throws IllegalArgumentException and interrupts normal construction.

The braces after require contain a lambda with no parameters: a small function that produces the message when the condition fails.

Replace 23 with 24 and run the program again. Construction fails and println is no longer executed.

Ranges and complete rules for Hora

CODE programas/C08_Intervalos.kt

```kotlin
class Hora(val horas: Int, val minutos: Int) {
    init {
        require(horas in 0..23 && minutos in 0..59) {
            "Invalid time."
        }
    }
}

fun main() {
    val partida = Hora(23, 40)
    println("${partida.horas}:${partida.minutos}")
}
```

OUTPUT

```
23:40
```

0..23 creates an inclusive range: it contains the integers from 0 to 23, including both endpoints.

horas in 0..23 asks whether horas belongs to the range. The result is true or false.

&& combines the rules for horas and minutos. With 23 and 40, both conditions are true and construction completes.

With 24 and 0, the condition fails. It also fails with 8 and 60. Both fields must be valid.

These rules are invariants of the model: every successfully constructed Hora object must satisfy them.

private restricts access to state

CODE programas/C09_Private.kt

```
01 class Contador {
02     private var valor = 0
03     fun avancar() {
04         require(valor < 3) { "Limit reached." }
05         valor += 1
06     }
07     fun atual(): Int = valor
08 }
09
10 fun main() {
11     val contador = Contador()
12     contador.avancar()
13     println(contador.atual())
14 }
```

OUTPUT

```
1
```

private allows access to valor only inside the class. The code in main cannot use contador.valor.

avancar and atual are public by default. They provide the operations that the rest of the program can use.

valor += 1 adds one and stores the result in valor. Here the values are restricted to the range from 0 to 3.

fun atual(): Int = valor has an expression body: the result is the value of the expression after =.

Encapsulation brings state and operations together, controlling access so that each change respects the established rules.

private set permits reading and controls changes

CODE programas/C10_PrivateSet.kt

```
01  class Emprestimo {
02      var ativo: Boolean = true
03          private set
04      fun devolver() {
05          require(ativo) { "Already returned." }
06          ativo = false
07      }
08  }
09
10  fun main() {
11      val emprestimo = Emprestimo()
12      println(emprestimo.ativo)
13      emprestimo.devolver()
14      println(emprestimo.ativo)
15  }
```

OUTPUT

```
true
false
```

The ativo property has a getter for reading and a setter for assignment. Kotlin provides these accessors automatically.

private set restricts only the setter. Outside the class, emprestimo.ativo remains available for reading.

Outside the class, emprestimo.ativo = false does not compile. Changes go through the devolver method.

require(ativo) accepts a return only while the loan is active. A second call is rejected before the assignment.

A property can also calculate its value

CODE programas/C11_Getter.kt

```kotlin
class Emprestimo {
    var ativo = true
        private set
    val devolvido: Boolean
        get() = !ativo
    fun devolver() {
        require(ativo) { "Already returned." }
        ativo = false
    }
}
fun main() {
    val emprestimo = Emprestimo()
    println(emprestimo.devolvido)
    emprestimo.devolver()
    println(emprestimo.devolvido)
}
```

OUTPUT

```
false
true
```

get() defines the code that executes when devolvido is read. = introduces the expression body of this getter.

! is logical negation: !true produces false, and !false produces true.

devolvido calculates its result from ativo. In this example, it does not store a second, independent Boolean.

Although devolvido is val, reads at different times can return different values because ativo can change.

val protects the reference, not the whole object

CODE programas/C12_ValReferencia.kt

```
01 class Livro(val id: String, var titulo: String)
02
03 fun main() {
04     val livro = Livro("L1", "Mensagen")
05     livro.titulo = "Mensagem"
06     println(livro.titulo)
07 }
```

OUTPUT

```
Mensagem
```

val livro prevents another reference from being assigned to the variable livro after initialisation.

The referenced object can have mutable properties. titulo is var, so livro.titulo = "Mensagem" is allowed.

id is val, so an external assignment to livro.id is not allowed.

The variable declaration and the property declarations control different things. val alone does not guarantee deep immutability.

Two variables can refer to the same object

CODE programas/C13_Referencias.kt

```
01 class Livro(val id: String, var titulo: String)
02
03 fun main() {
04     val primeiro = Livro("L1", "Mensagen")
05     val outroNome = primeiro
06     outroNome.titulo = "Mensagem"
07     println(primeiro.titulo)
08     println(outroNome.titulo)
09 }
```

OUTPUT

```
Mensagem
Mensagem
```

Only Livro("L1", "Mensagen") calls the constructor. The program creates one Livro object.

val outroNome = primeiro copies the reference. Both variables provide access to the same object.

The assignment through outroNome changes that object. Reading through primeiro observes the change.

To create another object, make another constructor call. Copying a reference does not perform that construction.

Returning text and displaying text

CODE programas/C14_RetornoApresentacao.kt

```kotlin
class Livro(val id: String, val titulo: String) {
    fun descricao(): String = "$id: $titulo"
    fun mostrar() {
        println(descricao())
    }
}

fun main() {
    val livro = Livro("L1", "Mensagem")
    val texto = livro.descricao()
    println("Record: $texto")
    livro.mostrar()
}
```

OUTPUT

```
Record: L1: Mensagem
L1: Mensagem
```

descricao returns a String. main can store it, add a prefix or use it in another operation.

mostrar calls descricao on the current object and prints the result. Its return type is Unit.

In println(descricao()), Kotlin first evaluates descricao() and passes the resulting String as the argument to println.

Separating calculation from presentation allows the same method to be reused with the console, a file or another form of interaction.

Playground: minutes since the start of the day

EXERCISE exercicios/C15_Hora.kt

```
class Hora(val horas: Int, val minutos: Int) {
    init {
        require(horas in 0..23 && minutos in 0..59) {
            "Invalid time."
        }
    }
    fun totalMinutos(): Int = TODO("Calculate")
}
fun main() {
    val hora = Hora(2, 30)
    println(hora.totalMinutos())
}
```

EXPECTED OUTPUT

```
150
```

Complete totalMinutos to return the number of minutes elapsed since 00:00. Keep the constructor and validation.

Each hour corresponds to 60 minutes. Add the minutes in the second field to the contribution from the hours.

Replace TODO("Calculate") with the required expression. TODO is a function that throws NotImplementedError when execution reaches it.

With Hora(2, 30), the result should be 150. Also test Hora(0, 0) and Hora(23, 59): expect 0 and 1439.

Finally, try Hora(24, 0). Validation should reject construction before totalMinutos is called.

Solution: each hour contributes 60 minutes

SOLUTION resolucoes/C16_Hora.kt

```kotlin
class Hora(val horas: Int, val minutos: Int) {
    init {
        require(horas in 0..23 && minutos in 0..59) {
            "Invalid time."
        }
    }
    fun totalMinutos(): Int = horas * 60 + minutos
}
fun main() {
    val hora = Hora(2, 30)
    println(hora.totalMinutos())
}
```

OUTPUT

```
150
```

horas * 60 converts the hours component into minutes. Multiplication is evaluated before addition.

For the example data, we obtain 2 * 60 + 30 = 120 + 30 = 150.

The expression body returns the calculated integer directly. We do not need to write return in this form.

This operation reads properties and returns a result. It neither changes the object nor prints anything inside the method.

With horas between 0 and 23 and minutos between 0 and 59, the result belongs to the range from 0 to 1439.

What a class should make explicit

Which objects does it represent? Livro describes a book copy, Hora describes a time of day, and Emprestimo describes a loan.

What data does it store? Parameters declared with val or var become properties. Further properties can appear in the body.

Which conditions does it guarantee? init and require check the data. Methods that control changes to state should preserve these rules.

Which operations does it offer? Methods such as totalMinutos and devolver describe what the rest of the program can ask the object to do.

How is it used? The constructor creates an instance. The dot accesses properties and calls accessible methods.

Equality in an ordinary class

CODE programas/D01_igualdade_class.kt

```
01 class Leitor(val nome: String)
02
03 fun main() {
04     val a = Leitor("Ana")
05     val b = Leitor("Ana")
06     val c = a
07     println(a == b)
08     println(a === b)
09     println(a === c)
10 }
```

OUTPUT

```
false
false
true
```

Complete program. The same text in two objects allows us to distinguish equality from identity.

a and b result from two constructor calls. They store the same name, but are two distinct objects.

c receives the reference held by a. The assignment neither constructs nor copies a Leitor.

=== asks whether the references designate the same object. == calls equals, while also handling null.

This class does not override equals. It inherits identity comparison from Any, the common base type of objects.

data class compares the declared data

CODE programas/D02_igualdade_dados.kt

```
01 data class Leitor(val id: String, val nome: String)
02
03 fun main() {
04     val a = Leitor("R1", "Ana")
05     val b = Leitor("R1", "Ana")
06     println(a == b)
07     println(a === b)
08     println(a)
09 }
```

OUTPUT

```
true
false
Leitor(id=R1, nome=Ana)
```

Complete program. We add data and explicitly choose the fields that form part of the value.

data modifies the class declaration. The compiler generates operations using the primary constructor properties.

The primary constructor is the list in parentheses after Leitor. Here it contains id and nome, both declared with val.

The fields of a and b match, so == returns true. The two constructions still produce distinct instances.

println(a) uses the generated textual representation. The field names make the contents of the object visible.

The operations generated by data

In these examples, we use the standard generated operations. equals, hashCode and toString can also have explicit implementations.

equals determines the equality used by ==. hashCode returns an Int used to organise lookups in sets and maps. Equal objects have the same hash code.

toString produces a description such as Leitor(id=R1, nome=Ana). println calls this operation when it receives the object.

copy constructs another instance. It allows new values to be supplied for some fields and reuses the remaining values from the original object.

component1, component2 and so on return the fields in constructor order. They underlie the destructuring declaration val (id, nome) = leitor.

The class needs at least one primary constructor parameter, and every such parameter must have val or var. We can also add methods and init.

copy constructs another object

CODE programas/D04_copy.kt

```
01 data class Hora(val horas: Int, val minutos: Int)
02
03 fun main() {
04     val original = Hora(8, 5)
05     val seguinte = original.copy(minutos = 6)
06     println(original)
07     println(seguinte)
08     println(original === seguinte)
09 }
```

OUTPUT

```
Hora(horas=8, minutos=5)
Hora(horas=8, minutos=6)
false
```

Complete program. This version isolates copy. We will add validation of the time in example D07.

minutos = 6 is a named argument: it selects the parameter by name rather than solely by position.

copy uses 6 for minutos and keeps 8 for horas because we have not supplied a new value for that field.

The call assigns nothing to original. The original object still represents 08:05.

seguinte refers to the new instance. === returns false because original and seguinte designate different objects.

Destructuring follows the order of the fields

CODE programas/D05_decomposicao.kt

```kotlin
data class Hora(val horas: Int, val minutos: Int)

fun main() {
    val hora = Hora(8, 5)
    val (h, m) = hora
    println(h)
    println(m)
    println(hora.component1())
}
```

OUTPUT

```
8
5
8
```

Complete program. Destructuring gives local names to the components of a value.

val (h, m) = hora declares two variables. The parentheses group the names that will receive the components.

h receives hora.component1(), corresponding to horas. m receives hora.component2(), corresponding to minutos.

The names h and m are local choices. The order of the fields in the constructor determines which values they receive.

Destructuring reads the object and leaves it unchanged. It neither removes the fields nor changes the time.

Properties declared in the body are left out

CODE programas/D06_propriedades_corpo.kt

```
01 data class Leitor(val nome: String) {
02     var consultas = 0
03 }
04
05 fun main() {
06     val a = Leitor("Ana")
07     a.consultas = 3
08     val b = a.copy()
09     println(a == b)
10     println(b.consultas)
11     println(a)
12 }
```

OUTPUT

```
true
0
Leitor(nome=Ana)
```

Complete program. The state of an instance may include properties that data does not take into account.

nome appears in the primary constructor. consultas appears in the body, between braces, with an initial value of zero.

The generated equality operation compares only nome. This is why a == b is true even though a has three consultations and b has none.

copy does not copy the current value of consultas. Constructing b runs the initialisation consultas = 0 again.

toString also omits consultas. Where a property is declared affects the generated operations.

Valid data with behaviour

CODE programas/D07_hora_validada.kt

```kotlin
data class Hora(val horas: Int, val minutos: Int) {
    init {
        require(horas in 0..23)
        require(minutos in 0..59)
    }
    fun totalMinutos(): Int = horas * 60 + minutos
}

fun main() {
    val hora = Hora(8, 5)
    println(hora.totalMinutos())
    println(hora.copy(minutos = 6).totalMinutos())
    // hora.copy(horas = 24)
}
```

OUTPUT

```
485
486
```

Complete program. We retain the Hora model from the travel project and add the validity rule for its domain.

A data class can have init blocks and methods. Validation takes place whenever an instance is constructed, including construction through copy.

horas in 0..23 tests membership of the inclusive interval. The condition on minutes applies the same principle between zero and 59.

totalMinutos returns horas * 60 + minutos. At 08:05, 480 + 5 = 485 minutes have elapsed since 00:00.

The commented line tests an error case. If you remove //, copy attempts to construct Hora(24, 5), and require rejects the argument.

Lists: reading and changing

CODE programas/D08_listas.kt

```
01 fun main() {
02     val nomes: MutableList<String> =
03         mutableListOf("Ana")
04     val leitura: List<String> = nomes
05     nomes.add("Rui")
06     println(leitura)
07     println(nomes === leitura)
08 }
```

OUTPUT

```
[Ana, Rui]
true
```

MutableList<String> means a mutable list whose elements are strings. <String> specifies the element type; it is not a comparison.

mutableListOf("Ana") constructs a list with one element. add("Rui") adds another element to the same object.

List<String> provides read operations. leitura receives the same list, but this type does not expose add.

A change made through nomes is visible through leitura. Read-only access does not guarantee that nobody can change the list.

val prevents the stored reference from being replaced. It does not prevent the mutation operations provided by MutableList.

copy retains references to the components

CODE programas/D09_copia_rasa.kt

```
01 data class Pasta(val nomes: MutableList<String>)
02
03 fun main() {
04     val a = Pasta(mutableListOf("Ana"))
05     val b = a.copy()
06     b.nomes.add("Rui")
07     println(a.nomes)
08     println(a === b)
09     println(a.nomes === b.nomes)
10 }
```

OUTPUT

```
[Ana, Rui]
false
true
```

Complete program. The output shows precisely which object is new and which object is shared.

The nomes property stores a reference to a list. copy constructs another Pasta and reuses that reference.

There are two folders, but only one list. b.nomes.add("Rui") changes the list that is also accessible through a.

a === b is false: the outer objects are different. a.nomes === b.nomes is true: the component is shared.

This operation is called a shallow copy. copy does not traverse the components to create independent copies of every object.

A separate list for the new folder

CODE programas/D10_lista_separada.kt

```
01 data class Pasta(val nomes: MutableList<String>)
02
03 fun main() {
04     val a = Pasta(mutableListOf("Ana"))
05     val outros = a.nomes.toMutableList()
06     val b = a.copy(nomes = outros)
07     b.nomes.add("Rui")
08     println(a.nomes)
09     println(b.nomes)
10     println(a.nomes === b.nomes)
11 }
```

OUTPUT

```
[Ana]
[Ana, Rui]
false
```

Complete program. Separating the lists is an explicit decision made when constructing the copy.

toMutableList() creates another list with the same elements. We pass this new list to the nomes parameter of copy.

b now holds a different list reference from the one held by a. Adding Rui to the list in b does not change the list in a.

The elements are still reused. Here they are String values, which provide no operations that change the text itself.

If the elements were mutable objects, the two lists could still share them. Copying the list separates only that level of the structure.

Equality depends on the meaning of the data

CODE programas/D11_significado_igualdade.kt

```kotlin
data class Fracao(val n: Int, val d: Int)

fun main() {
    val a = Fracao(1, 2)
    val b = Fracao(2, 4)
    println(a == b)
    println(a)
    println(b)
}
```

OUTPUT

```
false
Fracao(n=1, d=2)
Fracao(n=2, d=4)
```

This Fracao stores n and d without validation or simplification. The generated equality operation compares these two integers separately.

Mathematically, 1/2 and 2/4 represent the same rational number. The stored fields differ, so the program prints false.

To obtain equality of rational numbers, we can normalise the representation. In the fractions project, construction already performs this normalisation.

The same care is needed with book copies: the title does not identify a physical copy. The choice of fields should reflect the domain.

A data class can have mutable state

CODE programas/D12_dados_mutaveis.kt

```kotlin
data class Leitura(var valor: Int)

fun main() {
    val a = Leitura(10)
    val b = Leitura(10)
    println(a == b)
    a.valor = 11
    println(a == b)
    println(a.valor)
}
```

OUTPUT

```
true
false
11
```

Complete program. The kind of class declaration and whether its state can change are separate decisions.

var allows another value to be assigned to the property. The data modifier does not remove this possibility.

Initially, a and b have the same value. After the assignment a.valor = 11, equality compares 11 with 10.

val a preserves the reference stored in a. Assigning to the property changes the object designated by that reference.

For stable values, prefer val properties and components with no observable mutations. The shared list showed why val alone may be insufficient.

Playground: a duration in seconds

EXERCISE exercicios/D13_duracao_exercicio.kt

```kotlin
data class Duracao(val minutos: Int) {
    init { require(minutos in 0..1440) }
    fun segundos(): Int = TODO("Convert")
}

fun main() {
    val a = Duracao(50)
    val b = a.copy(minutos = 60)
    println(a.segundos())
    println(b.segundos())
    println(a.minutos)
    println(a == Duracao(50))
}
```

EXPECTED OUTPUT

```
3000
3600
50
true
```

Complete exercise with TODO. The displayed output is the result after the method has been completed.

Copy the complete program. Replace only TODO with an expression that converts minutes to seconds.

TODO("...") is a function that marks unfinished work. The program compiles, but this call throws NotImplementedError when executed.

Keep init: the permitted interval runs from zero to 1440 minutes. Use the instance property to calculate the answer.

Before running the program, justify the four expected results. Explain why copy does not change the 50 minutes stored in a.

Solution: a method on each instance

SOLUTION resolucoes/D14_duracao_resolucao.kt

```
01 data class Duracao(val minutos: Int) {
02     init { require(minutos in 0..1440) }
03     fun segundos(): Int = minutos * 60
04 }
05
06 fun main() {
07     val a = Duracao(50)
08     val b = a.copy(minutos = 60)
09     println(a.segundos())
10     println(b.segundos())
11     println(a.minutos)
12     println(a == Duracao(50))
13 }
```

OUTPUT

```
3000
3600
50
true
```

Complete solution. Also check Duracao(0) and Duracao(1440): they should produce 0 and 86400 seconds.

Each minute corresponds to 60 seconds. segundos() returns minutos * 60, using the minutes stored in the object on which it was called.

a.segundos() uses 50 and returns 3000. b.segundos() uses 60 and returns 3600.

copy constructs b with the named argument minutos = 60. The minutos field of instance a remains equal to 50.

The final comparison uses the data. a and Duracao(50) are distinct instances with the same number of minutes.

enum class fixes the alternatives

CODE programas/E01_enum_valores.kt

```kotlin
enum class DiaChegada { MESMO, SEGUINTE }

fun anunciar(dia: DiaChegada) {
    println(dia)
}

fun main() {
    val dia: DiaChegada = DiaChegada.SEGUINTE
    anunciar(dia)
    println(dia == DiaChegada.MESMO)
}
```

OUTPUT

```
SEGUINTE
false
```

Complete program. The constants are values of type DiaChegada. We do not construct new alternatives with DiaChegada().

DiaChegada is the type. MESMO and SEGUINTE are its two declared constants, separated by a comma.

DiaChegada.SEGUINTE selects one of these instances. We use the type name followed by a dot and the constant name.

The parameter accepts DiaChegada, not arbitrary text. A String such as "amanhã" cannot replace that argument.

The enumeration fixes the alternatives for the relative day of arrival. Each journey will still store its own times and duration.

Properties and methods of enum constants

CODE programas/E02_enum_propriedades.kt

```
01 enum class DiaChegada(val dias: Int, val rotulo: String) {
02     MESMO(0, "same day"),
03     SEGUINTE(1, "next day");
04 
05     fun minutosAdicionais(): Int = dias * 1440
06 }
07 
08 fun main() {
09     val dia = DiaChegada.SEGUINTE
10     println(dia.rotulo)
11     println(dia.minutosAdicionais())
12 }
```

OUTPUT

```
next day
1440
```

Complete program. Each constant is created with the values specified in its declaration.

The enum constructor declares dias and rotulo. MESMO(0, "same day") supplies the values for the first constant.

The semicolon ends the list of constants before the method declaration. Commas separate constants and arguments.

minutosAdicionais() reads dias from the constant on which it was called. One day corresponds to 24 * 60 = 1440 minutes.

rotulo is the description shown to the user. We can change it without changing the name SEGUINTE used in the code.

when produces a result for each case

CODE programas/E03_when.kt

```
01 enum class DiaChegada { MESMO, SEGUINTE }
02
03 fun descricao(dia: DiaChegada): String = when (dia) {
04     DiaChegada.MESMO -> "same day"
05     DiaChegada.SEGUINTE -> "next day"
06 }
07
08 fun main() {
09     println(descricao(DiaChegada.MESMO))
10     println(descricao(DiaChegada.SEGUINTE))
11 }
```

OUTPUT

```
same day
next day
```

Complete program. when is an expression: the value of the selected branch becomes the returned value.

when (dia) selects the branch whose value matches dia. The arrow -> separates the alternative from the resulting expression.

Both alternatives return String. The = sign after the signature makes when the result of the function descricao.

The branches cover all constants, so no else is needed. This expression is exhaustive: every DiaChegada has a result.

If we add a constant, this function will need a corresponding branch to continue covering all possible cases.

entries and for traverse the alternatives

CODE programas/E04_entries_for.kt

```kotlin
enum class DiaChegada { MESMO, SEGUINTE }

fun main() {
    val dias: List<DiaChegada> = DiaChegada.entries
    println(dias)
    for (dia in dias) {
        println(dia)
    }
}
```

OUTPUT

```
[MESMO, SEGUINTE]
MESMO
SEGUINTE
```

Complete program. This example uses entries, available in Kotlin 1.9 and later.

DiaChegada.entries provides the constants in declaration order. It is a specialised read-only list.

List<DiaChegada> specifies the type of the elements. The < and > signs delimit this type argument.

for (dia in dias) executes the body once per element. During each execution, dia denotes the current constant.

The first println displays the whole list. The println calls inside the loop display one constant per line.

Name, position and an explicit code

CODE programas/E05_name_ordinal.kt

```kotlin
enum class DiaChegada(val codigo: Int) {
    MESMO(10), SEGUINTE(20)
}

fun main() {
    val dia = DiaChegada.SEGUINTE
    println(dia.name)
    println(dia.ordinal)
    println(dia.codigo)
    println(DiaChegada.valueOf("SEGUINTE") == dia)
}
```

OUTPUT

```
SEGUINTE
1
20
true
```

name returns the constant name as a String. ordinal returns its position in the declaration, starting at zero.

The code 20 was chosen explicitly. If the constants are reordered, ordinal may change while codigo can stay the same.

Use a dedicated property when you need a stable identifier. The position of a constant tells you its current place in the order.

valueOf("SEGUINTE") looks up the exact name and returns the constant. A nonexistent name causes IllegalArgumentException.

Fixed atoms, constructed rules

CODE programas/E06_atom_rule.kt

```kotlin
enum class Atom { A, B, C, D, E }
data class Rule(
    val antecedent: Atom, val conclusion: Atom
) {
    fun accepts(atom: Atom): Boolean = atom == antecedent
}

fun main() {
    val regra = Rule(Atom.A, Atom.B)
    println(regra)
    println(regra.accepts(Atom.A))
    println(regra.accepts(Atom.C))
}
```

OUTPUT

```
Rule(antecedent=A, conclusion=B)
true
false
```

Complete program inspired by the modus ponens project. enum class and data class combine within the same model.

Atom provides five constants. Rule allows us to construct different rules using two of these constants as data.

antecedent and conclusion have type Atom. The order of arguments in Rule(Atom.A, Atom.B) defines the rule A implies B.

accepts receives an atom and compares it with the antecedent. The Boolean result indicates whether that atom matches the condition of the rule.

Constructing or inspecting a Rule does not prove B. A proof algorithm must still establish that the antecedent is available.

Playground: three possible days

EXERCISE exercicios/E07_enum_exercicio.kt

```
enum class DiaChegada { MESMO, SEGUINTE, DOIS_DIAS }

fun minutos(dia: DiaChegada): Int = TODO("Use when")

fun main() {
    for (dia in DiaChegada.entries) {
        println(minutos(dia))
    }
}
```

EXPECTED OUTPUT

```
0
1440
2880
```

Complete exercise with TODO. The displayed output is the target after the function has been implemented.

In this exercise, we deliberately extend the example to three alternatives. The original travel project still uses only two.

Replace TODO with a when expression that returns 0, 1440 or 2880, according to the value received.

Write one branch for each constant and omit else. The compiler should check that no alternative is missing.

The loop traverses the constants in declaration order. Predict the output before running the complete program.

Solution: every case has a result

SOLUTION resolucoes/E08_enum_resolucao.kt

```kotlin
enum class DiaChegada { MESMO, SEGUINTE, DOIS_DIAS }

fun minutos(dia: DiaChegada): Int = when (dia) {
    DiaChegada.MESMO -> 0
    DiaChegada.SEGUINTE -> 1440
    DiaChegada.DOIS_DIAS -> 2880
}

fun main() {
    for (dia in DiaChegada.entries) {
        println(minutos(dia))
    }
}
```

OUTPUT

```
0
1440
2880
```

Complete solution. Each constant is a fixed value, and the function describes how to use that value in the calculation.

The parameter dia supplies the value examined by when. Each branch associates a constant with the corresponding number of minutes.

The results are Int values. The expression body returns the selected number directly to the caller.

The for loop calls minutos three times. The sequence is MESMO, SEGUINTE, DOIS_DIAS, as given by entries.

The compiler knows the alternatives in the enumeration. Removing a branch makes this when incomplete and prevents compilation.

A rule shared by journeys

CODE programas/O01_RegrasViagem.kt

```
01 object RegrasViagem {
02     val limite: Int = 1440
03     fun aceita(minutos: Int): Boolean =
04         minutos in 0..limite
05 }
06 fun main() {
07     println(RegrasViagem.limite)
08     println(RegrasViagem.aceita(50))
09     println(RegrasViagem.aceita(1500))
10 }
```

OUTPUT

```
1440
true
false
```

The limit is common to all journeys. A single instance is enough to group these rules.

object RegrasViagem declares a type and makes its single instance available under that name.

limite is a property of the object. aceita is a method that returns Boolean.

RegrasViagem.aceita(50) calls the method directly on the object. We do not write RegrasViagem().

in 0..limite tests membership of the inclusive interval. A duration of 1500 minutes falls outside it.

The name gives access to the same instance

CODE programas/O02_Referencias.kt

```
01 fun main() {
02     val primeira = RegrasViagem
03     val segunda = RegrasViagem
04     println(primeira === segunda)
05     println(primeira.aceita(50))
06 }
```

OUTPUT

```
true
true
```

Fragment of main. The indicated file also includes RegrasViagem.

The name RegrasViagem is an expression denoting the shared instance.

Both val variables receive references to that same instance.

=== compares identity. The result true confirms that primeira and segunda denote the same object.

The object declaration is outside main. Calling a method still uses a dot and parentheses.

The state is shared as well

CODE programas/O03_ContadorPartilhado.kt

```
01 object ContadorViagens {
02     var total: Int = 0
03         private set
04     fun registar() {
05         total += 1
06     }
07 }
08 fun main() {
09     val a = ContadorViagens
10     val b = ContadorViagens
11     a.registar()
12     b.registar()
13     println(a.total)
14     println(b.total)
15 }
```

OUTPUT

```
2
2
```

a and b denote the same counter. Both method calls change the same total property.

In this case, total += 1 means total = total + 1. Each call adds one.

private set allows total to be read, but restricts assignment to the body of ContadorViagens.

This example follows two calls in a sequential execution. A new execution starts again at zero.

A class allows independent counters

CODE programas/O04_ContadoresIndependentes.kt

```
01 class Contador {
02     var total: Int = 0
03         private set
04     fun registar() {
05         total += 1
06     }
07 }
08 fun main() {
09     val a = Contador()
10     val b = Contador()
11     a.registar()
12     println(a.total)
13     println(b.total)
14 }
```

OUTPUT

```
1
0
```

Each call to Contador() constructs an instance. The class describes the state of each instance.

a.registar() changes only the counter denoted by a. Counter b remains at zero.

An object declaration provides one shared instance. A class lets us create as many instances as needed.

To count loans in independent libraries, each library can have its own counter.

The role of object in a project

The choice depends on how many instances the problem requires.

A class Exemplar allows each physical copy of a book to be represented with its own data.

An object RegrasViagem can group common operations that do not depend on a particular journey.

An object with var properties shares those properties across every place where it is used. That sharing is part of its behaviour.

An object declaration is initialised on first access. The safety of this initialisation does not automatically make concurrent changes to its state safe.

Playground: conversion with common rules

EXERCISE exercicios/O06_ConversorHora.kt

```kotlin
object ConversorHora {
    fun emMinutos(horas: Int, minutos: Int): Int {
        TODO("Validate and convert")
    }
}
fun main() {
    println(ConversorHora.emMinutos(2, 15))
    println(ConversorHora.emMinutos(23, 59))
}
```

EXPECTED OUTPUT

```
135
1439
```

Complete emMinutos. Accept hours from 0 to 23 and minutes from 0 to 59.

Replace TODO with two require calls and a return statement. The bounds are inclusive.

The answer should be horas * 60 + minutos. Each complete hour contributes 60 minutes.

Run the entire program in the Playground. Then change one hour value to 24 and observe its rejection.

TODO lets the starter program compile, but stops execution when called. It is not a valid answer.

Solution: validate before converting

SOLUTION resolucoes/O06_ConversorHora.kt

```kotlin
object ConversorHora {
    fun emMinutos(horas: Int, minutos: Int): Int {
        require(horas in 0..23)
        require(minutos in 0..59)
        return horas * 60 + minutos
    }
}
fun main() {
    println(ConversorHora.emMinutos(2, 15))
    println(ConversorHora.emMinutos(23, 59))
}
```

OUTPUT

```
135
1439
```

require tests a condition. If it is false, it throws IllegalArgumentException, and the method terminates with that failure.

Execution reaches return only after both validations succeed.

For 2 and 15, the expression calculates 2 * 60 + 15, which is 135.

We did not create an instance for each conversion. We called a method on the same ConversorHora twice.

An operation associated with the class

CODE programas/F01_FabricaDuracao.kt

```
01  class Duracao private constructor(val minutos: Int) {
02      companion object {
03          fun criar(minutos: Int): Duracao {
04              require(minutos in 0..1440)
05              return Duracao(minutos)
06          }
07      }
08  }
09  fun main() {
10      val viagem = Duracao.criar(50)
11      println(viagem.minutos)
12  }
```

OUTPUT

```
50
```

The factory function criar validates the argument and returns a new duration.

companion object declares an object inside Duracao. The class has a single, shared companion.

Duracao.criar(50) calls criar on that object. We do not yet need a duration instance.

private constructor restricts access to the constructor. The companion can call it from within the class.

viagem stores the returned instance. viagem.minutos reads the property of that duration.

Reading the constructor declaration

CODE programas/F01_FabricaDuracao.kt

```
01 class Duracao private constructor(val minutos: Int) {
02     companion object {
03         fun criar(minutos: Int): Duracao {
04             require(minutos in 0..1440)
05             return Duracao(minutos)
06         }
07     }
08 }
```

Fragment of the class. The complete file includes main.

class Duracao introduces the type of durations. private constructor explicitly specifies the visibility of the primary constructor.

val minutos: Int creates the property of each duration. The minutos parameter of criar receives the argument supplied in the call.

The : Duracao after the parentheses of criar specifies the result type.

In Duracao(minutos), the parentheses invoke the constructor. return delivers the new instance to the caller.

Tracing the call Duracao.criar(50)

The call returns a duration only after validating the data.

1. main calls criar on the companion of Duracao and passes the integer 50 to the minutos parameter.

2. require checks 50 in 0..1440. The condition is true, and execution continues.

3. Duracao(minutos) constructs an instance whose minutos property contains 50.

4. return delivers that instance. main binds it to the name viagem, and println displays 50.

With 1500, require fails at step 2. The call terminates with an exception and does not return a duration.

The companion and the instances are distinct

CODE programas/F04_InstanciasECompanion.kt

```
fun main() {
    val a = Duracao.criar(50)
    val b = Duracao.criar(50)
    val fabrica = Duracao.Companion
    println(a === b)
    println(fabrica === Duracao.Companion)
    println(fabrica.criar(20).minutos)
}
```

OUTPUT

```
false
true
20
```

Fragment of main. The indicated file includes the class Duracao.

Each execution of criar calls the constructor. a and b are two distinct duration instances, even though both contain 50.

When we omit the companion name, Kotlin calls it Companion. Duracao.Companion gives access to the object.

fabrica and Duracao.Companion denote the same object. There is one companion associated with the class.

fabrica.criar(20).minutos chains operations: it creates the duration and then reads minutos from the result.

Factory calls and instance calls

CODE programas/F05_Chamadas.kt

```
01 fun main() {
02     val d = Duracao.criar(50)
03     println(d.minutos)
04     // Duracao(50)       // private constructor
05     // Duracao.minutos   // an instance is required
06     // d.criar(50)       // criar belongs to the companion
07 }
```

OUTPUT

```
50
```

Excerpt from main. The complete file includes the declaration of Duracao.

Duracao.criar receives the data needed to produce a duration. It is accessed through the class name.

d.minutos reads the data of a particular duration. It is accessed through the reference d.

An instance property requires an instance. The class does not have a single minutos value shared by every instance.

The two characters // start a comment that continues to the end of the line. The compiler ignores the commented text.

The calculation needed for the time factory

CODE programas/F06_QuocienteEResto.kt

```
01 fun main() {
02     val total: Int = 95
03     val horas = total / 60
04     val minutos = total % 60
05     println(horas)
06     println(minutos)
07 }
```

OUTPUT

```
1
35
```

To represent 95 minutes as a time of day, we separate complete hours from the remaining minutes.

/ between integers computes an integer quotient. For the nonnegative values in this example, 95 / 60 is 1.

% computes the remainder of the division. Since 95 = 1 * 60 + 35, 95 % 60 is 35.

The results are Int values. The decimal number 1.5833... does not appear, because both operands are integers.

The next factory will accept only 0..1439. The resulting hours therefore lie in 0..23 and the minutes in 0..59.

Playground: a factory for a valid time

EXERCISE exercicios/F07_HoraDeMinutos.kt

```
class Hora private constructor(
    val horas: Int, val minutos: Int
) {
    companion object {
        fun deMinutos(total: Int): Hora {
            TODO("Validate and construct")
        }
    }
}
fun main() {
    val h = Hora.deMinutos(95)
    println(h.horas)
    println(h.minutos)
}
```

EXPECTED OUTPUT

```
1
35
```

Complete deMinutos to accept totals from 0 to 1439.

Replace TODO with require and return. Use total / 60 for the hours and total % 60 for the minutes.

The constructor call receives the two arguments in the order horas, minutos.

Also test 0, 60 and 1439. The expected times are 0:0, 1:0 and 23:59.

Test 1440 separately: it should be rejected, since it already corresponds to the beginning of the next day.

Solution: the factory preserves the bounds

SOLUTION resolucoes/F07_HoraDeMinutos.kt

```
class Hora private constructor(
    val horas: Int, val minutos: Int
) {
    companion object {
        fun deMinutos(total: Int): Hora {
            require(total in 0..1439)
            return Hora(total / 60, total % 60)
        }
    }
}
fun main() {
    val h = Hora.deMinutos(95)
    println(h.horas)
    println(h.minutos)
}
```

OUTPUT

```
1
35
```

The test total in 0..1439 validates the argument before division and construction.

For 95, the factory calculates the arguments 1 and 35 and calls Hora(1, 35).

The private constructor restricts construction to code inside the class. Users call deMinutos.

The val properties allow the fields to be read without assigning different values to them after construction.

Two responsibilities in the same example

Each instance represents a value. The companion provides a way to construct these values.

Hora represents a particular time, with the properties horas and minutos. Several times can coexist in the program.

Hora.deMinutos is a factory: it receives another representation, checks it and returns a valid instance.

The same principle can organise Fracao.criar(numerador, denominador), validating and normalising before returning the fraction.

A factory function need not always create a new instance. In the preceding examples it does, because it calls the constructor each time it runs.

Choosing a declaration

The three forms address different needs in a program.

class Hora: describes the type and allows several instances to be created, each with its own data.

object RegrasViagem: provides a named instance that groups shared operations or state.

companion object inside Hora: provides an object associated with the class, with members accessible as Hora.deMinutos.

The companion can exist without any instance of Hora. The properties horas and minutos still belong to the instances.

The contract of an operation

The same program can receive different operations.

An interface defines the members an object must provide to be used through that type.

The calculator needs an operation that takes two integers and returns an integer.

Addition and multiplication satisfy the same contract, although they perform different calculations.

The compiler checks the types and required members. Mathematical properties need justification and tests.

The signature of aplicar

CODE programas/I01_Classe_Implementa.kt

```
01 interface Operacao {
02     fun aplicar(a: Int, b: Int): Int
03 }
```

Excerpt from file I01. The complete program follows.

interface declares the type Operacao. The braces delimit the members that this type provides.

fun declares aplicar. a and b are parameters, both of type Int, separated by a comma.

Int after the parentheses specifies the result type. Each call must return an integer.

The declaration ends without a body. A concrete implementation must provide the calculation.

A class that implements the interface

CODE programas/I01_Classe_Implementa.kt

```kotlin
interface Operacao {
    fun aplicar(a: Int, b: Int): Int
}
class Soma : Operacao {
    override fun aplicar(a: Int, b: Int): Int {
        return a + b
    }
}
fun main() {
    val soma = Soma()
    println(soma.aplicar(4, 3))
}
```

OUTPUT

```
7
```

class Soma : Operacao declares a class that implements Operacao. Here the colon indicates a relationship between types.

Operacao appears without parentheses because the interface has no constructor to execute.

override marks the member that implements aplicar. return passes the sum back to the caller.

Soma() constructs an instance. soma.aplicar(4, 3) calls the method on that instance and produces 7.

Polymorphism: one function, two operations

CODE programas/I02_Polimorfismo.kt

```
01 interface Operacao {
02     fun aplicar(a: Int, b: Int): Int
03 }
04 class Soma : Operacao {
05     override fun aplicar(a: Int, b: Int): Int = a + b
06 }
07 class Produto : Operacao {
08     override fun aplicar(a: Int, b: Int): Int = a * b
09 }
10 fun mostrar(op: Operacao) {
11     println(op.aplicar(4, 3))
12 }
13 fun main() {
14     mostrar(Soma())
15     mostrar(Produto())
16 }
```

OUTPUT

```
7
12
```

mostrar receives op with type Operacao. It can receive any instance that implements this interface.

The body introduced by = returns the expression directly: a + b in Soma and a * b in Produto.

mostrar(Soma()) calls aplicar on the Soma instance. mostrar(Produto()) calls aplicar on the Produto instance.

mostrar uses the same call in both cases. The object it receives determines which implementation runs.

Tracing a call

Follow file I02 without changing the code.

In mostrar(Soma()), Soma() first constructs the object. Then mostrar receives the reference through the parameter op.

op.aplicar(4, 3) executes Soma.aplicar, which returns 7. println displays that result.

In the second call, op refers to an instance of Produto. The same arguments now produce 12.

The type Operacao determines which members are accessible through op. The object determines which body runs.

Call	Object received	Calculation	Output
mostrar(Soma())	Instance of Soma	4 + 3	7
mostrar(Produto())	Instance of Produto	4 * 3	12

An object can also implement an interface

CODE programas/I03_Object_Implementa.kt

```kotlin
interface Operacao {
    fun aplicar(a: Int, b: Int): Int
}
object Soma : Operacao {
    override fun aplicar(a: Int, b: Int): Int = a + b
}
object Produto : Operacao {
    override fun aplicar(a: Int, b: Int): Int = a * b
}
fun main() {
    val op: Operacao = Produto
    println(op.aplicar(4, 3))
}
```

OUTPUT

```
12
```

These operations do not need different data for each instance. object provides a single instance of each operation.

Produto refers directly to the object. We do not write Produto(), because we are not calling a constructor.

val op: Operacao = Produto stores the reference using the interface type.

The call op.aplicar(4, 3) executes the method of the object Produto. The contract remains the same.

Properties and methods with implementations

CODE programas/I04_Propriedades_Interface.kt

```kotlin
interface Rotulavel {
    val nome: String
    val rotulo: String
        get() = "Name: $nome"
    fun apresentar() {
        println(rotulo)
    }
}
class Livro(override val nome: String) : Rotulavel
fun main() {
    val livro = Livro("Mensagem")
    livro.apresentar()
}
```

OUTPUT

```
Name: Mensagem
```

val nome: String requires a readable property. The class Livro supplies it through override val nome.

rotulo has a getter: each access calculates the string using nome. The interface does not store a field for rotulo.

apresentar already has a body and can be used by Livro without rewriting the method.

An interface can require properties and provide calculations. Data storage belongs to the implementations.

An interface val can be implemented by var

CODE programas/I05_Val_Var_Interface.kt

```
interface Nomeado {
    val nome: String
}
class Leitor(override var nome: String) : Nomeado
fun main() {
    val leitor = Leitor("Ana")
    val consulta: Nomeado = leitor
    println(consulta.nome)
    leitor.nome = "Rita"
    println(consulta.nome)
    // consulta.nome = "Eva"  // Does not compile.
}
```

OUTPUT

```
Ana
Rita
```

Nomeado requires nome to be readable. Leitor also allows it to be changed, because it implements the property with var.

leitor and consulta refer to the same object. A change made through leitor becomes visible when consulta reads the name.

Through the type Nomeado, only the read access promised by val is available. The commented assignment does not compile.

The reverse does not work: an interface that requires var also needs write access, which val alone does not provide.

Playground: a subtraction operation

EXERCISE exercicios/I01_Diferenca.kt

```kotlin
interface Operacao {
    fun aplicar(a: Int, b: Int): Int
}
object Diferenca : Operacao {
    override fun aplicar(a: Int, b: Int): Int {
        TODO("Return a minus b")
    }
}
fun main() {
    val op: Operacao = Diferenca
    println(op.aplicar(9, 4))
    println(op.aplicar(4, 9))
}
```

EXPECTED OUTPUT

```
5
-5
```

Expected output after completing the method.

Paste only the indicated file into the Playground. Replace just TODO with the body that returns a minus b.

TODO marks unfinished code. If executed, it throws NotImplementedError and stops the program.

Keep the interface and the variable op of type Operacao. The solution must work through that type.

Explain why reversing the arguments changes the sign. The expected output appears below.

Solution: subtraction satisfies the contract

SOLUTION resolucoes/I01_Diferenca.kt

```
01 interface Operacao {
02     fun aplicar(a: Int, b: Int): Int
03 }
04 object Diferenca : Operacao {
05     override fun aplicar(a: Int, b: Int): Int {
06         return a - b
07     }
08 }
09 fun main() {
10     val op: Operacao = Diferenca
11     println(op.aplicar(9, 4))
12     println(op.aplicar(4, 9))
13 }
```

OUTPUT

```
5
-5
```

return a - b calculates the difference and returns an Int, satisfying the required result type.

In the first call, a receives 9 and b receives 4. The result is 5.

In the second call, a receives 4 and b receives 9. The result is -5.

We could also write override fun aplicar(a: Int, b: Int): Int = a - b.

The complete modular calculator

The next four slides form a single program.

The modulus n lies between 2 and 12. Each Long operand will be converted to a representative between 0 and n - 1.

The interface selects addition or multiplication. The calculator reduces the operands, applies the operation and reduces the result.

With n = 5, the operands -1 and 3 become 4 and 3. The sum 7 has representative 2.

The product 12 also has representative 2. Reducing first avoids multiplying the original Long operands directly.

Complete program, part 1 of 4

CODE programas/I06_Calculadora_Modular_Completa.kt

```
01 interface Operacao {
02     fun aplicar(a: Int, b: Int): Int
03 }
04 object Soma : Operacao {
05     override fun aplicar(a: Int, b: Int): Int = a + b
06 }
07 object Multiplicacao : Operacao {
08     override fun aplicar(a: Int, b: Int): Int = a * b
09 }
```

The contract and implementations of the two operations.

Operacao describes the call used by the calculator. The objects Soma and Multiplicacao provide the respective calculations.

a + b and a * b operate on Int values. The calculator will pass small representatives to these methods.

override confirms that each signature matches the member required by the interface.

Paste the four parts in the indicated order or use the complete file directly. There is only one main.

Complete program, part 2 of 4

CODE programas/I06_Calculadora_Modular_Completa.kt

```
01 class Modulo(val valor: Int) {
02     init {
03         require(valor in 2..12)
04     }
05     fun representante(x: Long): Int {
06         val n = valor.toLong()
07         return (((x % n) + n) % n).toInt()
08     }
09 }
```

A valid modulus and operand normalisation.

init runs during construction. require demands that valor belong to the inclusive interval 2..12.

toLong() converts the modulus to Long. Calculations involving x therefore use the same integer type.

In Kotlin, % calculates the remainder, which can be negative. We add n and calculate another remainder to obtain 0..n-1.

toInt() converts the final representative. This is safe because that number lies between 0 and 11.

Complete program, part 3 of 4

CODE programas/I06_Calculadora_Modular_Completa.kt

```
01 data class Resultado(
02     val a: Int, val b: Int, val valor: Int
03 )
04 class Calculadora(val modulo: Modulo) {
05     fun calcular(
06         op: Operacao, x: Long, y: Long
07     ): Resultado {
08         val a = modulo.representante(x)
09         val b = modulo.representante(y)
10         val bruto = op.aplicar(a, b)
11         val r = modulo.representante(bruto.toLong())
12         return Resultado(a, b, r)
13     }
14 }
```

Representing the result and coordinating the calculation.

Resultado groups the representatives used and the final value. Each calculation returns a new instance of this data class.

Calculadora receives and stores modulo in its constructor. calcular receives an Operacao and the Long operands x and y.

The variables a and b first receive the representatives. bruto receives the result of op.aplicar(a, b).

The final normalisation produces r. return Resultado(a, b, r) passes all the data back to the caller.

Complete program, part 4 of 4

CODE programas/I06_Calculadora_Modular_Completa.kt

```
fun main() {
    val calc = Calculadora(Modulo(5))
    val soma = calc.calcular(Soma, -1, 3)
    val produto = calc.calcular(Multiplicacao, -1, 3)
    println("Representatives: ${soma.a} and ${soma.b}")
    println("Sum: ${soma.valor}")
    println("Product: ${produto.valor}")
}
```

OUTPUT

```
Representatives: 4 and 3
Sum: 2
Product: 2
```

Modulo(5) constructs the modulus, which is then passed to the Calculadora constructor.

Soma and Multiplicacao are the objects passed as operations. The suffix L identifies literals of type Long.

${soma.a} reads a property inside an interpolated expression. The braces are part of string interpolation.

The program uses fixed data and works without keyboard input. Run the complete file in the Playground.

Tracing values and checking bounds

For x = -1 and n = 5, the expression first calculates -1 % 5 = -1, then 4 % 5 = 4.

Since n is positive, the first remainder lies between -(n - 1) and n - 1. Adding n does not exceed 23.

The representatives lie between 0 and 11. Their sum does not exceed 22 and their product does not exceed 121.

These bounds justify using Int in the two operations. A different implementation of Operacao would require checking its own bounds.

Operation, modulo 5	Representatives	bruto	r
Sum of -1 and 3	4 and 3	7	2
Product of -1 and 3	4 and 3	12	2

What the interface guarantees

The type expresses which members are available. The problem determines the expected behaviour.

Every implementation of Operacao has a method aplicar that takes two Int values and returns an Int.

The compiler does not prove that an object called Soma adds correctly. That rule requires reasoning and tests.

An implementation may depend on internal properties or use a different algorithm, provided that it respects the signature.

The calculator keeps operand normalisation separate from the choice of operation.

Using these declarations in the course projects

In the catalogue, classes control records and loans. Data types represent copies and readers.

In modular arithmetic, an interface allows operations to be selected. Stateless objects implement addition and multiplication.

For fractions, a factory in the companion can normalise values. For journeys, Hora and Duracao distinguish the roles of integers.

For modus ponens, enum represents atoms and data class represents rules. Each declaration addresses a specific need.

Types that allow absence

CODE programas/R01.kt

```
01 fun main() {
02     val texto: String? = null
03     val tamanho: Int? = texto?.length
04     val seguro: Int = tamanho ?: 0
05     println(tamanho)
06     println(seguro)
07 }
```

OUTPUT

```
null
0
```

String? allows a String or null. String, without ?, requires a non-null value.

null represents the absence of a value. Here, no text has been supplied.

?. accesses a member only when the value exists. If texto is null, texto?.length also returns null.

?: supplies an alternative when the left-hand side is null. The result of null ?: 0 is 0.

Replace null with "Ana": the outputs become 3 and 3. !! asserts that a value is present and may throw an exception. We prefer to handle absence explicitly.

Absence and early return

CODE programas/R02.kt

```kotlin
fun limparNome(texto: String?): String? {
    val recebido = texto ?: return null
    val limpo = recebido.trim()
    if (limpo.isBlank()) return null
    return limpo
}
fun main() {
    println(limparNome(" Ana "))
    println(limparNome("   "))
    println(limparNome(null))
}
```

OUTPUT

```
Ana
null
null
```

The function receives nullable text and can also return absence. The two ? symbols belong to different types.

If texto is null, ?: return null ends this call immediately. None of the subsequent lines in the function runs.

When text is present, trim() removes surrounding whitespace. isBlank() checks whether the String is empty or contains only whitespace.

if determines whether the function should reject the name. Braces can be omitted when the branch contains a single statement.

readlnOrNull() can supply a String? when reading the console. Here we use fixed arguments to reproduce the three cases in the Playground.

A rejected value can be handled

CODE programas/R03.kt

```
01 fun main() {
02     try {
03         val hora = 24
04         require(hora in 0..23) { "Invalid time" }
05         println("Accepted")
06     } catch (erro: IllegalArgumentException) {
07         println(erro.message)
08     }
09     println("Continuing")
10 }
```

OUTPUT

```
Invalid time
Continuing
```

try delimits the operation whose exception we want to handle. catch receives an exception of the indicated type.

require encounters a false condition and throws IllegalArgumentException. Execution jumps to the matching catch.

The message "Accepted" does not appear: after the failure, the program does not continue with the next line of the try block.

erro is the name of the catch parameter. erro.message reads the message. After catch, execution continues.

Handle the expected failure. An error message alone does not prevent a state change: validate before changing the state.

An object expression creates an anonymous object

CODE programas/R04.kt

```
01 interface Mensagem {
02     fun texto(): String
03 }
04 fun criar(): Mensagem = object : Mensagem {
05     override fun texto(): String = "Hello"
06 }
07 fun main() {
08     val a = criar()
09     val b = criar()
10     println(a.texto())
11     println(a === b)
12 }
```

OUTPUT

```
Hello
false
```

object : Mensagem { ... } is an expression. It creates an implementation of Mensagem without declaring a class name.

: Mensagem indicates the interface being implemented. override supplies the body of the method required by the interface.

Each call to criar evaluates the object expression again and constructs another instance.

Therefore a === b is false. The declaration object Regras, used in the earlier module, provides a shared instance.

The return type Mensagem exposes the interface operations. The caller can execute texto() without knowing the anonymous class.

Inheriting from a class

CODE programas/R05.kt

```
01 open class Documento(val titulo: String) {
02     open fun resumo(): String = titulo
03 }
04 class Relatorio(titulo: String) : Documento(titulo) {
05     override fun resumo(): String =
06         "Report: ${super.resumo()}"
07 }
08 fun main() {
09     val d: Documento = Relatorio("Library")
10     println(d.resumo())
11 }
```

OUTPUT

```
Report: Library
```

Ordinary classes cannot be inherited from by default. open allows subclasses to be created. open fun allows the method to be overridden.

: Documento(titulo) declares the superclass and calls its constructor with the argument received.

override marks the new definition of resumo. super.resumo() calls the superclass implementation.

The variable d has type Documento and refers to a Relatorio. The call executes the Relatorio implementation.

A class can inherit from only one class and implement several interfaces. Inheritance is appropriate when the new type is also a Documento.

An abstract class and a concrete implementation

CODE programas/R06.kt

```kotlin
abstract class Cartao(val nome: String) {
    abstract fun descricao(): String
}
class Leitor(nome: String) : Cartao(nome) {
    override fun descricao(): String = "Reader: $nome"
}
fun main() {
    val cartao: Cartao = Leitor("Ana")
    println(cartao.descricao())
}
```

OUTPUT

```
Reader: Ana
```

abstract class declares a class that cannot be instantiated directly. We can construct its concrete subclasses.

The abstract class can receive data in its constructor and store it, as with the property nome.

abstract fun declares a method without a body. A concrete subclass must provide the implementation using override.

Cartao(nome) calls the superclass constructor. The inherited property nome is available inside Leitor.

An interface specifies operations and can supply method bodies, but it has no fields of its own to store each instance's state.

Choose the declaration to suit the problem

class: several objects with data and operations. Example: a loan whose state changes through methods.

data class: values with equality and copying operations generated from constructor properties. Example: a logical rule.

enum class: known, enumerated alternatives. Example: MESMO_DIA and DIA_SEGUINTE.

object: a shared instance. Example: an addition operation with no separate data for each use.

companion object: an object associated with the class. Example: a factory that constructs normalised fractions.

interface: operations shared by different implementations. Example: applying a selected arithmetic operation.

Symbols with different roles

: after a name declares its type: nome: String. After a class name, it indicates a supertype: class A : I.

= initialises, assigns or introduces an expression body. == compares equality. === compares object identity.

() delimit parameters in a declaration and arguments in a call. {} delimit bodies, lambdas and expressions inside ${...}.

<T> indicates a type argument or type parameter in contexts such as List<String>. < between numbers expresses comparison.

? is part of a nullable type. ?. provides safe access. ?: selects an alternative when a value is absent.

-> separates a condition from its result in when. In a lambda with parameters, it separates the parameters from the body.

Documentation and further study

Syntax and functions: kotlinlang.org/docs/basic-syntax.html and kotlinlang.org/docs/functions.html

Classes and properties: kotlinlang.org/docs/classes.html and kotlinlang.org/docs/properties.html

Values and equality: kotlinlang.org/docs/data-classes.html and kotlinlang.org/docs/equality.html

Enumerations: kotlinlang.org/docs/enum-classes.html

Objects and interfaces: kotlinlang.org/docs/object-declarations.html and kotlinlang.org/docs/interfaces.html

Experiments: play.kotlinlang.org. After each exercise, change the data, predict the output and explain the result before running the program.